Discretisation invariance

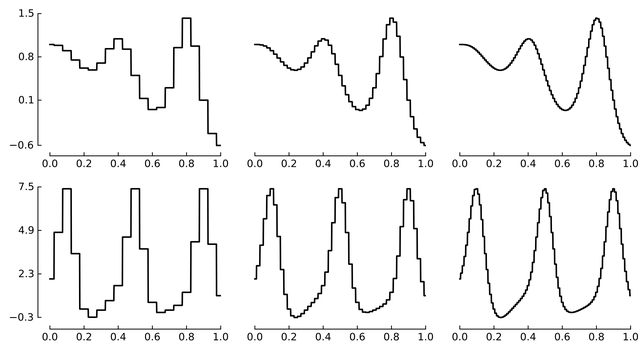
Vladimir Fanaskov[1,2], Ivan Oseledets[1,2,3]

[1]AXXX, [2]AIC, [3]INM

More examples and argument in blog-post: DeepONet, PCA-Net, Neural Fields, code.

What is discretisation invariance?

An example of a discretisation-invariant map $\psi = \mathcal{F}(\phi)$. When the input ϕ is available on a refined grid, the output ψ is refined as well.



- Main motivation: surrogate modelling for parametric PDEs.

$$\frac{\partial}{\partial \boldsymbol{x}_1}\left(k(\boldsymbol{x})\frac{\partial}{\partial \boldsymbol{x}_1}u(\boldsymbol{x})\right) + \frac{\partial}{\partial \boldsymbol{x}_2}\left(k(\boldsymbol{x})\frac{\partial}{\partial \boldsymbol{x}_2}u(\boldsymbol{x})\right) = f(\boldsymbol{x}),$$
$$\boldsymbol{x} \in \Gamma = (0,1)^2,\ u(\boldsymbol{x})|_{\boldsymbol{x}\in\partial\Gamma} = 0.$$

Typical problem: approximate the solution map

$$k(\boldsymbol{x}), f(\boldsymbol{x}) \to u(\boldsymbol{x}).$$

- To build discretisation-invariant architectures, **use continuous operations only:** inner products $(f,g) = \int (f(x))^\star g(x)dx$, differentiation $\frac{du(x)}{dx}$, composition operators $u(x + c(x))$, sup, inf, etc. *By avoiding discrete objects, the architecture becomes "discretisation-agnostic" by definition.*

An example of a discretisation-agnostic "architecture":

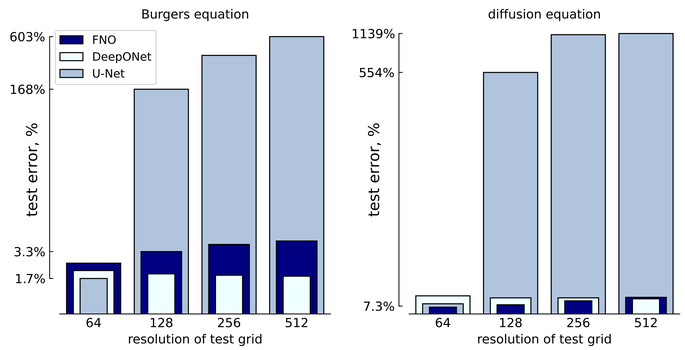
$f_0^{(i)}(\boldsymbol{x})$: input;

$$f_{k+1}^{(i)}(\boldsymbol{x}) = \sigma\left(\sum_j \int (\mathcal{K}_k(\boldsymbol{x},\boldsymbol{y}))^{(i,j)} f_k^{(j)}(\boldsymbol{y})d\boldsymbol{y} + \sum_j A^{(i,j)} f_k^{(j)} + b_k^{(i)}(\boldsymbol{x})\right);$$

$f_N^{(i)}(\boldsymbol{x})$: output.

To apply the equations above, additional details are required: (i) how to parametrise continuous objects (kernel, input/output, and bias), and (ii) how to compute the integral.

Numerical illustration



Fourier Neural Operator ✓

Fourier Neural Operator uses an integral kernel $u^{(i)}(x) = \sum_j \int (\mathcal{K}(\boldsymbol{x},\boldsymbol{y}))^{(i,j)} f^{(j)}(\boldsymbol{y})d\boldsymbol{y}$ based on spectral convolution:

1. $\hat{v}_k^{(j)} = \mathcal{F}(f^{(j)}(x))_k$, $k = 1, \ldots, k_{\max}$: FFT + **truncation**
2. $\hat{w}_m^{(i)} = \sum_j R_{ijm}\hat{v}_m^{(j)}$: linear transformation in Fourier space
3. $u^{(i)}(x) = \mathcal{F}^{-1}\left(\hat{w}^{(i)}\right)(x)$: inverse FFT

For a fixed discretisation, the linear transformation matrix is **block-circulant**; thus, it is equivalent to a standard convolution.

Arguments in favour of discretisation invariance

1. **The discretisation-invariant setup is theoretically natural.** The use of PDEs forces one to consider function spaces; thus, from a theoretical standpoint, it is natural to set up the learning problem in a Banach space.
2. **Multiscale training.** Discretisation invariance is a form of "weight-sharing across different input resolutions"; for many architectures, one can use downsampled data to pre-train a model and later fine-tune it with high-resolution data.
3. **Reduced inference cost.** In applications where a model must be evaluated repeatedly (e.g., sampling from diffusion models, guidance, MCMC, or energy-based models), one can resort to a coarse model most of the time.
4. **Hyperparameter optimisation.** Hyperparameter optimisation can be performed with the help of a crude model, which can later be retrained on data with the desired resolution.

PDEs are continuous and the architectures must be too. Models should be proxies for the underlying physics, not the grid.

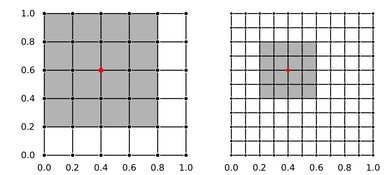
Vote for discretisation invariance!

Convolutional Neural Networks ✗

Standard convolutions (especially dilated and with residuals) are widely used for PDEs discretised on the uniform structured grids

$$u^{(i)}(x_m) = \sum_p \sum_j K_p^{ij} f^{(j)}(x_{m-p})$$

The receptive field of a vanilla convolution explicitly depends on the input resolution; therefore, it is not discretisation-invariant.



Arguments against discretisation invariance

1. **Discretisation invariance is an asymptotic condition.** Formally, discretisation invariance is only strictly achieved when the number of grid points tends to infinity.
2. **Only finite-resolution data is available.** Training, evaluation, and deployment are always performed in a fixed-resolution setting. A good practical example is weather forecasting based on the ERA5 dataset.
3. **No generalisation is observed when the grid is refined.** There is no "zero-shot super-resolution," and one should not expect an architecture to generalise beyond the resolution seen during the training stage.
4. **Discretisation invariance does not guarantee good performance.** Any architecture can be made discretisation-invariant with a suitable choice of encoder (e.g., scalar product, sampling) and decoder (e.g., interpolation, finite series as in DeepONet). Yet, the performance of such architectures can differ vastly.

I do not care about the continuum limit. Performance on the target resolution is the only metric that matters.

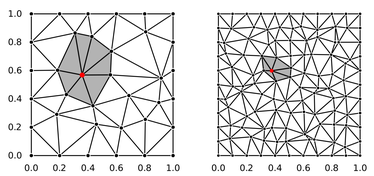
Vote against discretisation invariance!

Graph Neural Networks ✗

Most often the grid is unstructured, so a natural alternative to convolution is the GNN:

$$u^{(i)}(x_m) = \sum_{k\in N(m)} \sum_j K_k^{ij} f^{(j)}(x_k)$$

When the grid is refined, the receptive field in the spatial domain x space shrinks the same way as for the convolutional networks.

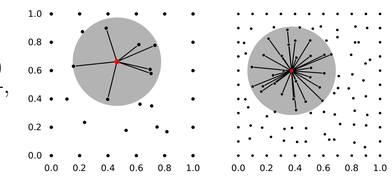


Graph Kernel Networks ✓

A Graph Kernel Network is a discretisation-invariant version of a GNN:

$$u^{(i)}(x_m) = \frac{\sum_{k\in N(m)} \sum_j K^{ij}(x_m, x_k) f^{(j)}(x_k)}{|N(m)|},$$

with two changes: (i) the sum is replaced by an average, and (ii) the graph of neighbours is dynamically recomputed after refinement.



LaTeX TikZposter